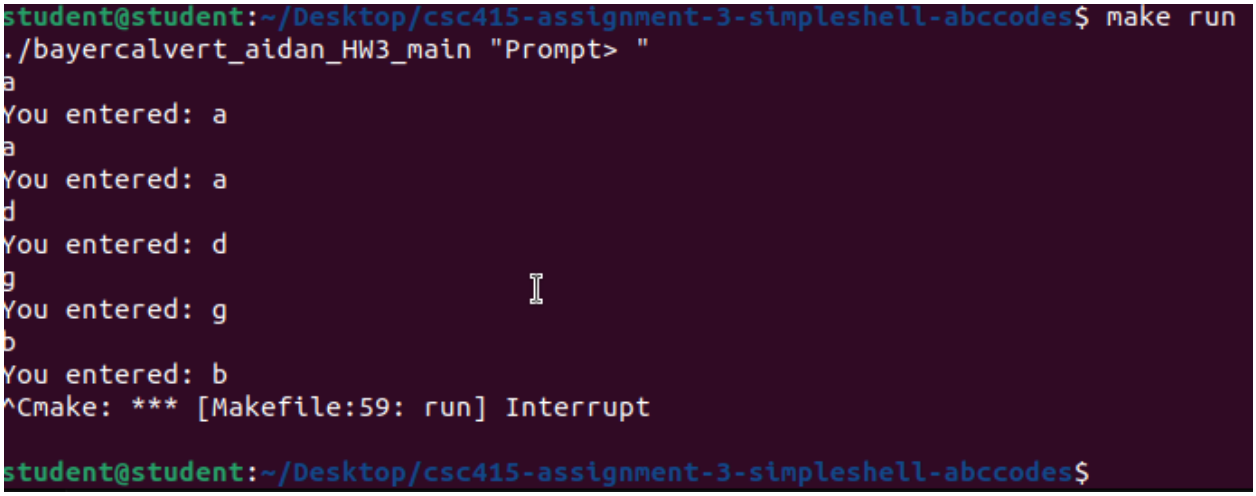
Aidan Bayer-Calvert
Github: abccodes
ID: 922119403
CSC415-03 Operating Systems

Assignment 3 - Simple Shell with Pipes

Description:
This assignment is about a simple shell which allows users to input commands, execute them, and supports chaining multiple commands by using pipes all while handling various inputs and errors.

Approach:
First I cloned the assignment and read through the GitHub instructions a couple of times. I then pulled up the slides talking about the simple shell for reference. This was helpful as it references some important concepts such as forking.

To get started, I first just want to make sure everything is working correctly, so I printed hello world by using printf, including stdio.h to access the printf library. After this, the next logical step in my opinion was to create a loop going over all lines of user input and printing them to the console. At first I thought to use the argv array, but realized since we are creating a shell it would need to be continuously running or reading input from the user and we aren't reading arguments but rather input text. My next approach was to look up the C library/functionality that can read input from the command line. After a quick Google, I learned that the function fgets is used to read input and can limit the number of characters read as well. It also handles space and when a user inputs control+d it returns null, meaning we can exit the program. So using fgets I created a while loop that would continuously run as long as the fgets input is not null. The fgets function took in a pointer to an array of chars which is basically where the input is being stored, the buffer size(103 bytes specified in instructions) which is the maximum number of characters to read at one time, and stdin which basically specifies where fgets reads its input from, in this case the standard input. This appeared to be working with no issues (screenshot below).



After this, I moved on to understanding what I should do for the next steps, I also committed my code as a good practice to save my work. I figured the next best logical step was to tokenize the inputs (not worrying about piping for now), so every different input is split by its spaces, so I can

see what the user inputted arguments are and easily access them. To do this, I first conducted research on methods for splitting a string and landed on the strtok function of the string.h library. This function breaks a string into smaller segments based on a specified delimiter. In my case, the delimiter was a space which allows me to distinguish between separate command arguments. The function operates by successively retrieving tokens until none are left, returning NULL when there is nothing left to extract. To structure the tokenization process, I allocated an array to store pointers to each extracted token and introduced an index variable to track the position of each command in the array. I then initialized strtok by passing the user input string and the space delimiter, allowing the function to locate the first token. After figuring out the basic logic for tokenizing the string into commands, I integrated it into the main program structure by setting up a while loop that continuously reads user input, processes it, and then breaks it down into separate arguments. Since we need to repeatedly handle commands, I enclosed this logic in the while loop that runs indefinitely until the termination condition is met. Additionally, I made sure to properly terminate the array of commands by assigning NULL to the last position. I also realized that strtok() modifies the original input string by replacing each delimiter with '\0', meaning that once tokenization is complete, the input array no longer contains the original user input but instead a sequence of separated strings. I had to ensure that index started at 0 so that the first token was not skipped, preventing an off-by-one error. Another detail I accounted for was handling cases where the user input consisted only of spaces or an empty string, which could cause strtok() to return NULL immediately. By checking if commands[0] was NULL before proceeding, I ensured that no unnecessary processing occurred for empty input. Finally, to confirm that tokenization was functioning correctly, I printed out each extracted token, verifying that arguments were correctly split and stored in the array.

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
ls cd
You entered: ls cd
Terminal
[cd
]
command_1 command_2 command_3 command_4
You entered: command_1 command_2 command_3 command_4
Tokens:
[command_1]
[command_2]
[command_3]
[command_4
]
^Cmake: *** [Makefile:59: run] Interrupt

student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

After this, I moved on to actually creating the logic to run the commands. I needed a way to create a new process for each user command and replace that process with the requested command. Through my research(slides/google/class/instructions), I determined that the best

approach was to use fork() to create a child process, execvp() to execute the command within that child process, and waitpid() to ensure the parent process properly waits for the child to complete before continuing. The first thing I did was implement the fork function into my code. fork() basically creates an extra child process from the parent, and both continue executing independently. The child is responsible for executing the command, while the parent waits for it to complete before then accepting some new input. To distinguish between parent and child, I checked the return value of fork(), a return value of 0 indicated the child process, a negative value indicated a failure to create a child process, and a positive value represented the parent process. If fork() failed, I printed an error message and continued to the next iteration of the loop. Inside of the child process I used the execvp function to use the user's command. What execvp does in this case is replace the child process with a new program specified by users input. It requires an argument array, which I had already structured during making my command tokenization, making the integration pretty straightforward. What I did is call execvp and if it failed(maybe command was invalid), it returned -1, allowing me to print an error message. before terminating the child process with exit(EXIT_FAILURE). Since execvp() does not return if successful (as it replaces the process image), any code after execvp() within the child process would only execute in case of failure. In the parent process, I called waitpid to pause the execution until the child process finished. This made sure that if there were multiple processes, they would not run in parallel and that the shell maintained a structured execution flow. I also learned how to extract the child's exit status using WEXITSTATUS(status) after looking at Google, which helped me to print status messages about the command's termination. One difficulty I thought of when doing this was handling edge cases where the user entered an empty command. If just using execvp, it would attempt to execute an invalid command, resulting in an error. To fix this, I added a check ensuring that commands[0] was not NULL before attempting execution. Next I added support for the exit command. Initially, "exit" was being treated as a regular command, which was not working because when calling execvp to attempt to execute "exit" it was leading to an error. To resolve this, I added a condition before forking that checked if the user entered "exit" so that the shell printed an exit message and broke out of the main shell loop gracefully with no errors. I also re-read all the non-piping directions to make sure I included everything. When reading the directions talking about handling EOF, I realized I wasn't handling it correctly in my code. I adjusted my code so that if my shell encounters an error while reading a line of input, it will report the error using perror and then exit using EXIT_FAILURE. The function feof(stdin) checks whether the end-of-file (EOF) indicator has been set for standard input, which occurs when the user presses Ctrl+D, ensuring the program exits cleanly when no further input is expected. The perror function prints a system-defined error message corresponding to the last encountered error. The exit(EXIT_FAILURE) function terminates the program immediately with a failure status, signaling that an error has occurred. I also realized I didnt add functionality for if a user entered an empty line. When testing and entering an empty line there would be no feedback, I added functionality to check if a user input was empty by checking the length of the input using strlen() and checking if it was equal to 0. If it was then I would print an error and continue or skip over this input. Now, with command execution functioning properly, the basics of my shell were set up and I was able to move on to piping. I also committed this to GitHub. A screenshot of some of my various tests is below:

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
pwd
/home/student/Desktop/csc415-assignment-3-simpleshell-abccodes
Child 12337 exited with status 0
ls
bayercalvert_aidan_HW3_main    bayercalvert_aidan_HW3_main.o  Makefile
bayercalvert_aidan_HW3_main.c  commands.txt                   README.md
Child 12339 exited with status 0
/bin/ls
bayercalvert_aidan_HW3_main    bayercalvert_aidan_HW3_main.o  Makefile
bayercalvert_aidan_HW3_main.c  commands.txt                   README.md
Child 12340 exited with status 0
/usr/bin/whoami
student
Child 12341 exited with status 0
doesnotexist
Execution failed: No such file or directory
Child 12342 exited with status 1
ls -l
total 56
-rwxrwxr-x 1 student student 17360 Feb 27 16:55 bayercalvert_aidan_HW3_main
-rw-rw-r-- 1 student student  4839 Feb 27 16:55 bayercalvert_aidan_HW3_main.c
-rw-rw-r-- 1 student student 10632 Feb 27 16:55 bayercalvert_aidan_HW3_main.o
-rw-rw-r-- 1 student student    66 Feb 24 16:59 commands.txt
-rw-rw-r-- 1 student student  1871 Feb 25 23:59 Makefile
-rw-rw-r-- 1 student student  7317 Feb 24 16:59 README.md
Child 12343 exited with status 0
echo This is a test
This is a test
Child 12344 exited with status 0
exit
Exiting shell...
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

My next step was to implement piping. To start, I first wanted to calculate how many pipes the user entered so I could assign resources accordingly when processing commands. To do this, I iterated through the user's input string and counted th | character and how many times it appeared. This allowed me to determine how many commands were involved in the pipeline because the commands would always be the amount of pipes + 1. I also added a debug statement to print the number of pipes detected to ensure this step was working correctly. After testing with different inputs (both with and without pipes), I confirmed that the program correctly counted the number of pipes in each command string. Next, I needed to split the user input into different command segments based on the | delimiter. To achieve this, I allocated an array pipeSegments[numPipes + 1] to store each individual command as a separate string. To split the input into separate commands, I used the strtok function from the string.h library with "|" as the delimiter to split up the user input by the pipes. This basically breaks the input string into segments and removed each | but replaces with \0, treating each command as an independent string with a null pointer. Once I tested this, I quickly realized that leading and

trailing spaces needed to be removed as well because that could cause issues later on. When entering the | symbol its more intuitive for a user to enter command | command than command|command so this is a good step to include. To trim spaces, I implemented manual logic by removing leading spaces by using a while loop with the condition (*segment == ' '), then incrementing segment++, which moves the pointer forward until it reaches a non-space character and removing trailing spaces by moving to the end of the string and replacing trailing spaces with null terminators (\0). This was done using a pointer end initialized to the last character of the segment, which moved backward until it found a non-space character. Once the this backward and forward space removal was complete, I stored the cleaned-up string directly in pipeSegments[pipeIndex++]. After successfully separating the commands, I printed them for debugging to verify by running different tests like single commands, commands with multiple arguments, and multi-stage pipings, which helped confirm that everything was working as expected.

Next, I processed each command further by splitting the split pipes it into individual arguments using spaces as delimiters. To achieve this it was similar logic to what I had previously done in my shell without pipes. I iterated over each command stored in the pipeSegments array and created an array to hold the individual arguments for that command. I initialized an index argIndex to track the number of arguments stored. For each command segment, I used the strtok function again with " " (a space) as the delimiter. This helpd break up the command string into separate tokens representing the command and its arguments. The first call to strtok(pipeSegments[i], " ") extracts the command na, and subsequent calls to strtok(NULL, " ") continue extracting tokens until there are no more left which then returns NULL. Each extracted argument is stored in args[argIndex++]. Once all arguments are processed, I append NULL to the args array to properly terminate it. I printed the arguments to verify correctness. To do this, I iterated through args and printed each stored argument in brackets to visually confirm correct parsing as shown in screenshot below.

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
Prompt$ ls
Number of pipes: 0
Parsed Commands:
[ls]
Command 0 Arguments:
[ls]
Prompt$ ls -l
Number of pipes: 0
Parsed Commands:
[ls -l]
Command 0 Arguments:
[ls] [-l]
Prompt$ ls -l | grep .c | wc -l
Number of pipes: 2
Parsed Commands:
[ls -l]
[grep .c]
[wc -l]
Command 0 Arguments:
[ls] [-l]
Command 1 Arguments:
[grep] [.c]
Command 2 Arguments:
[wc] [-l]
Prompt$ ^Cmake: *** [Makefile:59: run] Interrupt

student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

The next step was to actually run the commands using pipes. First I did a lot of research and tests some implementations from what I was learning but was having a lot of difficulties getting started. At this point all of my code was in my main file as well, so getting started was confusing me because of how disorganized my code was. I decided it was in my best interest to refactor my code into different functions so I can better understand what parts I need to modify to execute this final step in the project.

To begin, I reviewed my code to find all the most important sections that could be easily refactored into standalone functions. The goal was to make main a more high-level main function with a lot of supporting smaller functions. I decided to introduce three functions: getUserInput(), splitCommands(), and parseArguments(). First, I extracted input handling into getUserInput(), moving all logic related to reading user input, checking for termination conditions, and handling errors out of main(). This function now manages the shell prompt, calls fgets() to retrieve input, and ensures that errors or EOF conditions are properly handled. Next, I refactored command splitting into splitCommands(), which takes the input string, counts the number of pipes, and splits the string into separate commands while trimming unnecessary spaces. Next, I extracted argument parsing into parseArguments(), which tokenizes each

command using spaces as delimiters and stores the arguments in an array formatted for execution with execvp().

After this, I moved on to the actual final part of the project which is running the commands with piping. The first step was to set up the pipes correctly. I first did a lot of research on the pipe functions and what piping is because I have never before used it. I learned that each pipe connects two commands, the number of pipes needed is equal to numPipes calculated previously by iterating over the input and looking for the | character. Each call to the pipe function creates a pair of file descriptors, [0] for reading, and [1] for writing. I had to do a lot of googling to figure out that the file descriptors start at 3 and 4 because standard input, standard output, and standard error already occupy file descriptors 0, 1, and 2. This means the first pipe will have read = 3 and write = 4, the next one will be read = 5 and write = 6, and so on, incrementing as more pipes are created. Once the pipes set up, I need to fork a new process for each command so within the child processes, I redirected input and output as needed using dup2 function. If a command was not the first one in the pipeline, I redirected its input from the read end of the previous pipe. If it was not the last command, I redirected its output to the write end of the current pipe. This chained the commands together so that the output of one process would become the input of the next. After setting up redirections, I used execvp to replace the child process with the actual command, I also added some error handling just incase the execvp function had failed, it would print an error and exit the child process with the exit function. The parent process after forking a child, closed the write end of the pipe immediately since the child process already took care of redirections. Finally, after all commands were forked and running, I used waitpid() in the parent process to ensure all child processes completed before returning to the prompt.

After successfully setting up the core logic for handling pipes, I moved on to finalizing the shell. I tested various inputs and re-read the instructions, one of the first mistakes I encountered was not handling exit in my new version of code with piping. To fix this I basically just implemented the exact same logic I previously had where I check if a user enters “exit” and if so gracefull exit the shell by breaking the main loop. Another mistake I realized was not printing the pid and status after each command. Very similar to the “exit” I basically just copied and pasted this logic from my previous version as I had already done it there. My last step was to create a .h file to include my function prototypes as well as libraries as I think it increases the readability.

Overall, this project gave me a way deeper understanding of C and allowed me to get familiar in depth with a lot of new concepts such as forking, piping, functions, vectors, exit(), perror(), and dup(). After this I uploaded all my code to github and turned in the assignment.

Issues and Resolutions:
I had a bunch of issues along the way, but for the sake of simplicity and readability, I included just a couple of the many issues I faced.

One issue I encountered was when I was refactoring my code into functions. When doing this I made some mistakes which led to this error below:

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
bayercalvert_aidan_HW3_main.c: In function 'main':
bayercalvert_aidan_HW3_main.c:42:12: warning: implicit declaration of function 'getUserInput' [-Wimplicit-function-declaration]
   42 |         if(getUserInput(input, bufferSize) == -1) {
      |            ^~~~~~~~~~~~
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
Prompt$  ^Cmake: *** [Makefile:59: run] Interrupt

student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

I declared a function getUserInput but it was not being recognized in my code. I realized I needed to add the function header before main in order for main to be able to recognize that the function was there. After this it fixed my issue(see below).

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
Prompt$ ^Cmake: *** [Makefile:59: run] Interrupt

student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

Another one of the issues I encountered was when I was trying to make my function split commands that splits all the commands by pipes.

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
bayercalvert_aidan_HW3_main.c: In function 'splitCommands':
bayercalvert_aidan_HW3_main.c:177:9: error: 'numPipes' redeclared as different kind of symbol
  177 |     int numPipes = 0;
      |         ^~~~~~~~
bayercalvert_aidan_HW3_main.c:174:59: note: previous definition of 'numPipes' with type 'int *'
  174 | void splitCommands(char *input, char **pipeSegments, int *numPipes) {
      |                                                      ~~~~~^~~~~~~~~
bayercalvert_aidan_HW3_main.c:181:14: error: invalid type argument of unary '*' (have 'int')
  181 |             (*numPipes)++;
      |              ^~~~~~~~~
make: *** [Makefile:50: bayercalvert_aidan_HW3_main.o] Error 1
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

This error came from re-declaring my numPipes inside of my splitcommands function. I was already declaring numPipes as a global variable in my main function then passing in a pointer to that variable into my function so redclaring numPipes variable in my function confused the compiler as it was already defined. I removed my declaration and this fixed the issue.

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run
./bayercalvert_aidan_HW3_main "Prompt> "
prompt$ exit
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

One issue I had was when testing one of the final versions of my code, promt$ seemed to display too many times when doing make run < commands.

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run < commands.txt
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
./bayercalvert_aidan_HW3_main "Prompt> "
bayercalvert_aidan_HW3_main    bayercalvert_aidan_HW3_main.o  Makefile
bayercalvert_aidan_HW3_main.c  commands.txt                   README.md
"Hello World"
total 80
drwxrwxr-x 4 student student  4096 Feb 28 20:53 .
drwxr-xr-x 6 student student  4096 Feb 24 16:59 ..
-rwxrwxr-x 1 student student 18848 Feb 28 20:53 bayercalvert_aidan_HW3_main
-rw-rw-r-- 1 student student  9750 Feb 28 20:53 bayercalvert_aidan_HW3_main.c
-rw-rw-r-- 1 student student 15000 Feb 28 20:53 bayercalvert_aidan_HW3_main.o
-rw-rw-r-- 1 student student    64 Feb 28 19:23 commands.txt
drwxrwxr-x 8 student student  4096 Feb 27 17:46 .git
-rw-rw-r-- 1 student student  1871 Feb 25 23:59 Makefile
-rw-rw-r-- 1 student student  7317 Feb 24 16:59 README.md
drwxrwxr-x 2 student student  4096 Feb 25 23:38 .vscode
    PID TTY          TIME CMD
  20455 pts/0    00:00:00 bash
  20507 pts/0    00:00:00 make
  20514 pts/0    00:00:00 sh
  20515 pts/0    00:00:00 bayercalvert_ai
  20519 pts/0    00:00:00 ps
     64     304
ls: cannot access 'foo': No such file or directory
prompt$ prompt$ prompt$ prompt$ prompt$ prompt$ prompt$ Exiting shell
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

My first steps to solve this was to move my printf prompt$ to my main function, I thought maybe my function was somehow being called over and over again. This didnt do anything, so I moved on to other solutions. I then tried to run the commands.txt commands myself to see if prompt$ was still displaying multiple times. I noticed it wasnt so it made me think maybe something to do with running commands.txt was the issue. I thought the issue might be with not checking if EOF was reached properly so I added a check using !feof(stdin) then display $prompt. This also did not fix anything so I moved on. I realized that when using < commands.txt, the program was still printing prompt$, even though no user interaction was happening so it was not an actual super serious issue as it would only appear with files not if a user was typing in the termina. To fix this I basically used a bandaid approach by using the isatty(STDIN_FILENO), which checks if the standard input is coming from a terminal. By wrapping my printf("prompt$ ") statement inside an if (isatty(STDIN_FILENO)) condition, the prompt only prints when running in terminal preventing it from appearing repeatedly when using redirected input.

Screen shot of compilation:

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make clean
rm *.o bayercalvert_aidan_HW3_main
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make
```

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make
gcc -c -o bayercalvert_aidan_HW3_main.o bayercalvert_aidan_HW3_main.c -g -I.
gcc -o bayercalvert_aidan_HW3_main bayercalvert_aidan_HW3_main.o -g -I. -l pthread
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```

Screen shot(s) of the execution of the program:

```
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$ make run < commands.txt
./bayercalvert_aidan_HW3_main "Prompt> "
bayercalvert_aidan_HW3_main    bayercalvert_aidan_HW3_main.o  Makefile
bayercalvert_aidan_HW3_main.c  commands.txt                   README.md
Child 22266 exited with status 0
"Hello World"
Child 22267 exited with status 0
total 80
drwxrwxr-x 4 student student  4096 Feb 28 21:43 .
drwxr-xr-x 6 student student  4096 Feb 24 16:59 ..
-rwxrwxr-x 1 student student 19184 Feb 28 21:43 bayercalvert_aidan_HW3_main
-rw-rw-r-- 1 student student 10601 Feb 28 21:37 bayercalvert_aidan_HW3_main.c
-rw-rw-r-- 1 student student 15920 Feb 28 21:43 bayercalvert_aidan_HW3_main.o
-rw-rw-r-- 1 student student    64 Feb 28 21:43 commands.txt
drwxrwxr-x 8 student student  4096 Feb 27 17:46 .git
-rw-rw-r-- 1 student student  1871 Feb 25 23:59 Makefile
-rw-rw-r-- 1 student student  7317 Feb 24 16:59 README.md
drwxrwxr-x 2 student student  4096 Feb 25 23:38 .vscode
Child 22268 exited with status 0
    PID TTY          TIME CMD
  20455 pts/0    00:00:00 bash
  22263 pts/0    00:00:00 make
  22264 pts/0    00:00:00 sh
  22265 pts/0    00:00:00 bayercalvert_ai
  22269 pts/0    00:00:00 ps
Child 22269 exited with status 0
     64     304
Child 22270 exited with status 0
Child 22271 exited with status 0
ls: cannot access 'foo': No such file or directory
Child 22272 exited with status 2
student@student:~/Desktop/csc415-assignment-3-simpleshell-abccodes$
```